

# VeriGate-FW: Verifiable AI-Gated Firmware Updates for IoT Devices with Revocable Analysis Models on a Blockchain

Author  
Department, Institution  
email

**Abstract**—Over-the-air firmware updates are the main channel through which IoT fleets are both repaired and attacked. Signature-based frameworks such as Uptane answer *who* produced an update but not whether a genuine update is *safe* to install, and systems that add a risk analysis treat the analyser as trusted infrastructure that is never wrong. We present VeriGate-FW, a software-only update gate that combines a deterministic cryptographic stage (eight fail-closed checks against on-chain publisher, firmware and model registries) with an AI risk stage (an SBOM vulnerability model and a firmware-image anomaly model, both exported to ONNX and registered by hash) whose scores are combined by an on-chain policy into approve / defer / reject. Every verdict names the model hashes that produced it, commits to its feature vector and is anchored on-chain in Merkle batches; an analysis model is therefore a revocable supply-chain artefact: revoking its hash marks every verdict it produced stale and the gateway re-verifies the affected devices with the successor. A local language model writes a rationale that is stored but never consulted by the decision. On a laptop with a local Hardhat chain, the deterministic gate takes 89 ms per verification, the full gate 111 ms in-process and 216 ms per device over HTTP, anchoring costs 1048 gas per verdict at 200 verdicts per batch (99.5 % below one transaction per verdict), a revoked model’s verdicts for 50 devices are replayed in 11.7 s, and eleven scripted attacks are caught in 100 % of 52 runs. We report the detector’s failures with the same care: the anomaly model catches appended payloads (AUROC 1.0) but small patches and section swaps are at chance, and the learned SBOM model improves calibration (MAE 0.17 vs 1.01) without improving ranking over a CVSS/EPSS/KEV baseline.

**Index Terms**—firmware update, IoT security, blockchain, IPFS, SBOM, anomaly detection, model governance

## I. INTRODUCTION

A vendor with tens of thousands of deployed devices cannot visit them; it ships firmware over the air. The receiving device must decide whether the file is authentic, untampered, not an old version replayed by an attacker, not withdrawn by the vendor, and—even when all of that holds— not quietly shipping a library with a serious known vulnerability. The first four questions are answered by signed metadata and a revocation record; frameworks such as The Update Framework and Uptane [1], [2] and blockchain-anchored variants [3], [4], [5] do this well. The fifth question is where those systems stop, and it is where the recent SBOM-triage literature [7] starts—without any binding between the analysis and the update decision.

VeriGate-FW closes that gap with three properties that, to our knowledge, no prior system combines:

- 1) **Two-stage gate with three outcomes.** A deterministic stage that can only reject or defer, followed by a risk stage whose output is thresholded by an *on-chain* policy into approve, defer (human review) or reject. The AI can tighten a decision, never loosen it.
- 2) **Accountable verdicts.** Each verdict records the hashes of the models that produced it and a hash of the quantised feature vector, is signed by the gateway and anchored as a Merkle leaf on-chain, so anyone holding the model files can recompute and check it.
- 3) **Revocable models.** A model is registered by hash like a firmware release. Revoking it marks every batch that used it stale; gateways swap to the registry’s successor and re-verify the affected devices automatically.

A language model produces a human-readable rationale for reviewers; its content is stored by content identifier in the verdict and is never read by the decision path.

Everything is measured, and the failures are reported alongside the successes (Section VI). The implementation, models, fixtures, attack scripts and raw results are released.

## II. RELATED WORK

TUF [1] and Uptane [2] define role-separated signed metadata with rollback and freeze protection; they have no notion of an analysis decision and no cross-vendor audit trail. FOTB [3] and Baza et al. [4] anchor firmware fingerprints on a blockchain and serve images from IPFS; DIDAuth-IoTFW [5] authenticates firmware with decentralised identifiers and verifiable credentials on real hardware. LedgerGuard [6] adds SBOMs, staged roll-out and Merkle-rooted accountability of device receipts on a permissioned ledger. The SBOM-triage pipeline of Tolay [7] scores firmware SBOMs with CVSS, EPSS and CISA KEV but does not connect the score to an update decision or an audit trail. None registers the analysis model, none can revoke it, and none combines a deterministic gate with a three-outcome policy. The supplementary comparison table ([docs/comparison.md](#)) lists feature presence per system as described in each source.

### III. DESIGN

#### A. Actors and registries

A *publisher* holds an Ed25519 key bound to a DID in `PublisherRegistry` (register, rotate, revoke, reputation). A release is a signed *manifest*—firmware hash, SBOM hash, semantic version, device model, expiry, IPFS CIDs, publisher DID—whose canonical JSON (a JCS subset without floats) is signed; the release identifier is the keccak-256 of the unsigned canonical manifest and is registered in `FirmwareRegistry`, which enforces monotonic versions per publisher and device model, a future expiry and the publisher role. Artefacts are content-addressed on IPFS with CIDv1 computed identically to Kubo. `ModelRegistry` holds analysis models by SHA-256 hash with a status and an optional successor. `PolicyContract` holds the weights and thresholds; `VerdictRegistry` holds batch roots and answers `staleByModel`. All contracts are Solidity 0.8.26 on OpenZeppelin `AccessControl` with custom errors and 100 % statement coverage.

#### B. Stage 1: the deterministic gate

Eight pure checks run in order and short-circuit: firmware hash, manifest signature under the DID’s registered key, publisher active, version strictly above the device’s installed version, expiry, SBOM hash, registry record not revoked, and every configured model active. Any failure except expiry yields REJECT; an expired manifest yields DEFER with an alert (the freeze attack). A dependency outage (chain RPC or IPFS) yields DEFER, never APPROVE. Because the checks are pure functions of recorded inputs, a verdict can be replayed.

#### C. Stage 2: the AI risk gate

**SBOM risk.** Ten integer-quantised features (component and CVE counts, CVSS statistics, EPSS mass, KEV count, outdatedness, dependency age) are computed from a CycloneDX SBOM with OSV, a dated EPSS snapshot and the CISA KEV catalogue from a disk cache. A `HistGradientBoostingRegressor` predicts the growth of the release’s exposure between a feature date  $T_0$  and a label date  $T_1$ , where exposure is  $\sum_c \text{EPSS}_{T_1}(c)$ , the expected number of exploited CVEs;  $r_{\text{sbom}} = 1 - \exp(-\hat{y}/6)$ . The corpus is 624 real OpenWrt release manifests across 49 releases; the split is temporal (newest 20 % held out) [8]. **Image anomaly.** Fourteen pure features (chunk entropy statistics, printable ratio, ELF header and program-header sanity, declared size, appended bytes, and deltas versus the previous release) feed an `IsolationForest` trained on 165 benign OpenWrt ELF binaries only;  $r_{\text{img}}$  maps the anomaly score so that the benign median is 0 and the benign 99th percentile is 0.5. A documented, seeded mutation catalogue (byte-patch, append, section-swap, pack, downgrade-relabel) produces the synthetic tampered set used for evaluation only [9]. Both models run only as ONNX at inference; training is byte-deterministic so the registered hashes are meaningful; SHAP attributions of the top three features are recorded per verdict.

#### D. Policy, reputation and verdicts

$R = w_{\text{sbom}}r_{\text{sbom}} + w_{\text{img}}r_{\text{img}} + w_{\text{rep}}(1 - \rho)$  in basis points with weights (0.4, 0.4, 0.2) and thresholds  $\tau_{\text{approve}} = 0.45$ ,  $\tau_{\text{reject}} = 0.70$  chosen so that the three genuine fixture releases sit at approve / defer / approve with neutral reputation [11]. Reputation  $\rho$  is an on-chain EWMA ( $\alpha = 0.1$ ) raised by signed install receipts and lowered by release-level rejections; only the gateway role may write it, and with  $w_{\text{rep}} = 0.2$  it can move a release into DEFER but never into REJECT by itself. The verdict record (release, device, model hashes, feature hash, scores,  $R$ , verdict, rationale CID, timestamp) is signed by the gateway (EIP-191), hashed into a domain-separated Merkle leaf and committed in batches bounded by size or time; the gateway serves proofs and the contract verifies them.

#### E. Model revocation

An administrator revokes a model hash naming a successor. The gateway polls model status; on revocation it reads `staleByModel`, joins the batch identifiers with its local batch store (only where the Merkle root matches the chain), locates the successor by *hash* under its model directory, swaps only that scorer slot and re-runs the full gate for every stale (release, device) pair through the normal batcher. Without a successor file the revoked hash stays configured and Stage 1 rejects on *model active*: the gate fails closed [12].

#### F. Explainer

A local `qwen2.5:3b-instruct` via Ollama receives the SBOM diff against the previous release, the scores and the top attributions and must answer with a schema-constrained JSON object (summary, up to five risks, a suggested action). The validated object is pinned to IPFS and its CID stored in the verdict. Release-level verifications wait for it; device polls never do; a missing or malformed answer degrades to “no rationale”, never to “no verdict” [10].

### IV. IMPLEMENTATION

Python 3.11 (FastAPI gateway, asyncio device emulator, Typer CLIs, structlog), Solidity/Hardhat 2.29, a React dashboard typed from the gateway’s OpenAPI schema, and Docker Compose for portability (`make up` starts the chain, IPFS, deployer, model registrar, gateway, fleet and dashboard; `make smoke` proves a verdict from a clean machine). The fixtures are real OpenWrt `busybox` builds (22.03.7, 23.05.3, 24.10.0 and an end-of-life 19.07.10) with SBOMs derived from the real package manifests, so Stage-2 inputs are in the training distribution. Eleven attack scripts run only against the local emulated fleet. The code base has 331 unit tests (92 % coverage), 72 contract tests (100 %), 12 integration and 12 end-to-end tests, all in CI.

### V. EVALUATION

All measurements are on one laptop (Intel i7-1255U, 15 GB RAM, no GPU) against a local Hardhat node and Kubo, with the explainer disabled unless stated;  $\geq 5$  repetitions, warm-up runs discarded, median with interquartile range; every run

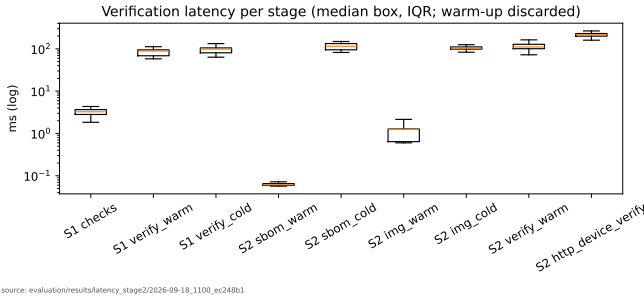


Fig. 1. Verification latency per stage (ms, log scale); source directories in the figure footer.

directory records the commit, machine, contract addresses, model hashes and vulnerability-snapshot date. Figures are produced only by `evaluation/figures.py`.

#### A. Latency

Fig. 1. The eight Stage-1 checks on a fetched bundle take 3.3 ms (IQR 2.8–3.7); the full deterministic verification path—model status and policy reads from the chain, checks, record signing and batch queueing—takes 88.7 ms warm and 98.3 ms when the manifest, firmware and SBOM are fetched from IPFS first (`latency_stage1/2026-09-18_1045`,  $n = 90$  per metric). Stage 2 adds 115 ms for SBOM scoring and 104 ms for image scoring the first time a release is seen and 0.1 ms / 1.3 ms afterwards (per-hash caches); a complete verification for a device costs 110.8 ms in-process and 216 ms over HTTP (`latency_stage2/2026-09-18_1100`,  $n = 60$ ). The explainer is three orders of magnitude slower: six sequential rationales took 42.5–91.3 s each (median 88.4 s) on the CPU, which is why devices never wait for it.

#### B. Anchoring cost

Fig. 2. `commitBatch` costs 209 642 gas independent of the number of verdicts it covers (the root, the count and two model hashes are stored; leaves are not), so the cost per verdict falls from 209 642 with one verdict per transaction to 20 964 at 10, 4 193 at 50, 2 096 at 100 and 1 048 at 200—a 99.5 % reduction (`gas_per_verdict_vs_batched/2026-09-18_1044`, five receipts per size). These are `gasUsed` values from local receipts; no gas price and no L2 data-availability cost are included. On a real rollup each transaction also pays for its calldata, which batching reduces in the same direction but not by the same factor, so the figure above is a statement about execution gas alone; a public-testnet measurement is left for future work.

#### C. Model revocation propagation

Fig. 3. After the gateway notices a revocation, replaying 5 / 20 / 50 stale device verdicts with the successor takes 7.5 / 8.8 / 11.7 s (IQR within 0.1 s), to which the 2 s poll interval adds the detection bound (`revocation_propagation/2026-09-18_1109`, five repetitions per size). About 7 s is the fixed cost of loading the successor ONNX model and its SHAP background; each

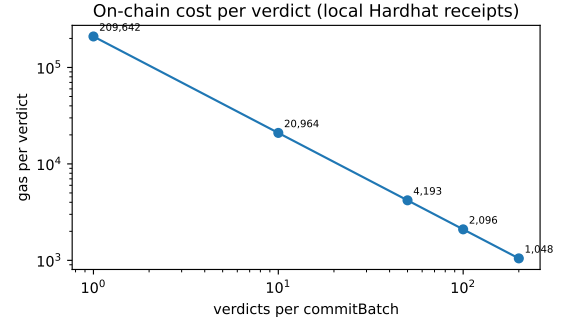


Fig. 2. Gas per anchored verdict versus batch size (local Hardhat receipts).

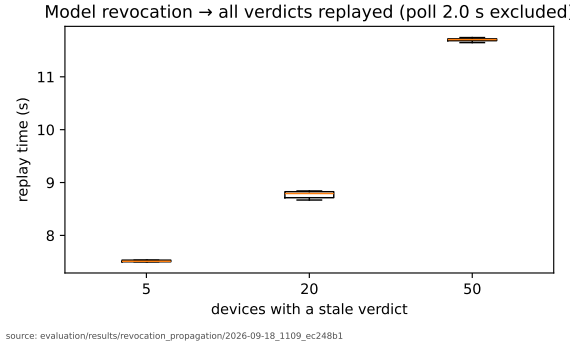


Fig. 3. Time from revocation detection to the last replayed verdict versus fleet size.

further device adds roughly 90 ms. Every replayed verdict kept its outcome because the successor in this experiment is the same detector retrained with another seed—the experiment measures propagation, not detection.

#### D. Attack matrix

Table I. Each scenario was run five times against the running gateway (`attack_matrix/2026-09-18_1114`); *poisoned-model* twice, because a model hash can be revoked once and the shipped stack has one primary image model and one successor (the second run revokes the successor with no replacement and every replayed verdict fails closed). All 52 runs produced the expected outcome.

#### E. Detector quality

**Image anomaly** (Fig. 4, `detection_f1/2026-09-18_1125`, 165 benign and 165 mutated binaries per class, operating point  $r_{\text{img}} \geq 0.5$ ): appended payloads are detected with recall 1.000, precision 0.988 and AUROC 1.000; packing has AUROC 0.895 but recall 0.061 at the operating point; byte patches (0.012 / 0.543), section swaps (0.012 / 0.504) and relabelled downgrades (0.000 / 0.431) are at or below chance. The benign false-positive rate of 1.2 % is optimistic because the benign rows are the training rows. **SBOM risk** (`sbom_ranking/2026-09-18_1125`, 196 held-out rows from the 14 newest releases, 200 bootstrap resamples):

TABLE I  
ATTACK MATRIX: EXPECTED VS OBSERVED OUTCOME, RUNS AND PASS RATE.

| attack                                | expected / observed | runs | pass |
|---------------------------------------|---------------------|------|------|
| tamper (patched image behind the CID) | REJECT              | 5    | 100% |
| forge (attacker key, victim DID)      | REJECT              | 5    | 100% |
| stolen key (revoked key)              | REJECT              | 5    | 100% |
| rollback (older genuine release)      | REJECT              | 5    | 100% |
| freeze (expired manifest)             | DEFER               | 5    | 100% |
| SBOM swap (clean hash, dirty SBOM)    | REJECT              | 5    | 100% |
| vulnerable-but-genuine (EOL base)     | DEFER               | 5    | 100% |
| hidden payload (200 KiB appended)     | DEFER               | 5    | 100% |
| bad history (four prior rejects)      | DEFER               | 5    | 100% |
| policy tamper (non-admin setPolicy)   | BLOCKED             | 5    | 100% |
| poisoned model (revoke, replay)       | REPLAYED            | 2    | 100% |

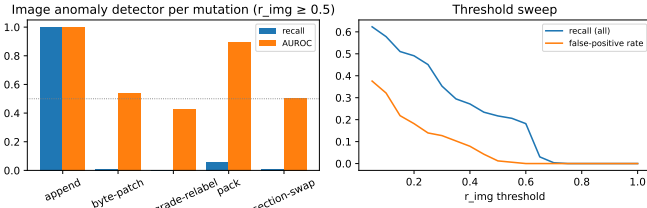


Fig. 4. Image anomaly detector per mutation class and threshold sweep.

Spearman 0.965 for the model versus 0.976 for the CVSS/EPSS/KEV baseline, both with AUROC 1.0 for “≥ 3 expected exploited CVEs”; the model’s gain is calibration—MAE 0.167 versus 1.012 expected exploited CVEs—which is what absolute policy thresholds need, not ranking.

## VI. LIMITATIONS

Each item is backed by a measured failure (docs/limitations.md). (i) The image detector sees *structural* tampering only; a malicious build that preserves size, entropy and layout passes Stage 2, and detection of an appended payload depends on having the previous release: without a predecessor the same payload scores 0.41 and the release would be approved. (ii) No component of the OpenWrt corpus matches the CISA KEV catalogue, so the KEV feature and the baseline’s KEV floor are untested on real data; the vulnerable-but-genuine scenario shows EPSS/CVSS mass instead. (iii) The learned SBOM model does not out-rank the baseline. (iv) The explainer costs 43–91 s per rationale on a CPU; the full demonstration takes 348 s with it and 53 s without. (v) A model hash can be revoked once, so replay scenarios consume models. (vi) Release-level rejections burn the publisher’s reputation and the listener’s own verification counts as a second signal, so the order of scenarios matters. (vii) Because the release identifier is the manifest hash, a squatter can register a predictable manifest first; the squatted copy fails Stage 1 but the genuine publisher must re-issue with a fresh expiry. (viii) Gas is local `gasUsed` only; the fleet is emulated; all timings are from one laptop; and the

first Stage-2 verification on an empty vulnerability cache takes about ten minutes.

## VII. CONCLUSION

VeriGate-FW shows that an AI risk gate for firmware updates can be made accountable with the same machinery that makes the firmware itself accountable: register the model by hash, name it in every verdict, anchor the verdicts, and revoke the model when it is wrong. The deterministic stage keeps the AI from ever loosening a decision, the on-chain policy keeps the rules auditable, and the explain-only language model keeps the reviewer informed without touching the outcome. The measured costs—about 100 ms per verification, about a thousand gas per anchored verdict, seconds to replay a revoked model’s verdicts—are compatible with fleet-scale operation; the measured weaknesses of the detectors define the work that remains.

## REFERENCES

- [1] J. Samuel, N. Mathewson, J. Capps, R. Dingledine, “Survivable key compromise in software update systems,” in *Proc. ACM CCS*, 2010.
- [2] T. K. Kuppusamy, A. Brown, S. Awwad, D. McCoy, R. Bielawski, C. Mott, S. Lauzon, A. Weimerskirch, J. Capps, “Uptane: Securing software updates for automobiles,” in *Proc. ESCAR Europe*, 2016; Uptane Standard, <https://uptane.org>.
- [3] A. Yohan, N.-W. Lo, “An over-the-blockchain firmware update framework for IoT devices,” in *Proc. IEEE DSC*, 2018.
- [4] M. Baza, M. Nabil, N. Lasla, K. Fidan, M. Mahmoud, M. Abdallah, “Blockchain-based firmware update scheme tailored for autonomous vehicles,” in *Proc. IEEE WCNC*, 2019.
- [5] W. M. A. B. Wijesundara, J.-S. Lee, E. Aloupogianni, D. Tith, H. Suzuki, T. Obi, “DIDAuth-IoTFW: Decentralized firmware authentication for smart home IoT devices using verifiable credentials,” *Internet of Things*, vol. 34, 101788, 2025.
- [6] LedgerGuard, “Permissioned-DLT firmware governance, artifact anchoring, staged rollout control, and Merkle-rooted update accountability,” <https://github.com/ajalkhodair-spec/ledgerguard>, 2026.
- [7] A. Tolay, “Automated SBOM-driven vulnerability triage for IoT firmware: A lightweight pipeline for risk prioritization,” arXiv:2601.01308, 2026.
- [8] VeriGate-FW model card models/sbom\_risk.card.md, 2026.
- [9] VeriGate-FW model card models/image\_anomaly.card.md, 2026.
- [10] VeriGate-FW ADR-0002, “The LLM explains; deterministic models decide,” 2026.
- [11] VeriGate-FW ADR-0008, “Default policy parameters,” 2026.
- [12] VeriGate-FW ADR-0009, “Model revocation: successor by hash, replay at the gateway,” 2026.